

AI 시대, 빠르고 안전한 앱 프로토타입

새도우 IT를 양지로 올리는 법

장민석 · KISTI 과학기술연구망센터 · msjang@kisti.re.kr

출연연 오픈소스 테크데이 2026 · 2026. 10. 7.



msjang.github.io/ostday26-sadp

이용 안내 / License

- 이 발표자료는 © 2026 장민석, CC BY-NC-ND 4.0
- 제3자 자료(기관 공지 화면, 신문 기사, 미러 목록 등)는 비평·인용 목적으로 사용했으며 각 권리자에게 저작권이 있음
- SADP 코드는 별도 오픈소스 라이선스로 공개

오늘 40분

- 1부 · 새도우 IT — 12분
- 2부 · 하기 쉬운 실수, 안전하게 운영하는 법 — 6분
- 3부 · SADP — 7분
- 4부 · 손잡이 — 7분
- 5부 · 정책 제안 — 8분

i 제품 발표가 아닙니다.

앞 12분은 과거, 가운데 13분은 기술, 뒤 15분은 제도 이야기입니다

? 세션 소개에는 SADP 소개라고 썼는데, 오늘은 앞뒤를 더 말씀드립니다
— 왜 이게 필요했는지(1·2부)와 플랫폼 하나로 안 되는 것(5부).
SADP 자체는 7분입니다. 40분 중 7분이면 짧은 게 맞다고 봅니다

1부 · 새도우 IT

0:00 - 0:12

승인 없이 만들어진 것들.
예전에도 있었고, 금지가 이겼고,
그 결과가 지금 여러분 기관입니다.

새도우 IT Shadow IT 가 뭐냐면

i 부서의 승인, 인지 또는 감독 없이
엔터프라이즈 네트워크에서 사용되는 모든 소프트웨어·하드웨어

- **PoC** 개념검증, 사전검증 는 성격상 새도우에 가깝습니다.
승인 전에 만들어보야 승인 여부를 정할 수 있으니까요
- 즉 새도우 IT는 예외가 아니라, 모든 새 시스템이 거쳐가는 단계입니다

⚠ 그러면 질문이 바뀝니다
— "막을 것인가"가 아니라 "어떻게 양지로 올릴 것인가"

2001년 9월 20일



공지사항

바이러스 경보(님다)

등록일	2001-09-20	전화번호	042-481-4661
작성자	전산2	조회수	10557

바 이 러 스 경 보

올해 최악의 바이러스로 평가될 님다(Nimda)가 e-메일을 통해 급속도로 퍼지고 있어 전 세계가 비상에 걸려있습니다.

이 바이러스의 특징은 메일을 읽기만 해도 감염된다고 합니다.

"readme.exe" 딸린 메일 읽지마세요.
"첨부파일에 readme.exe 가 들어있는 e-메일은 바로 지워야 합니다."

【 예방 】

1. 본인이 원하지 않은 메일이거나 또는 확인할 수 없는 제목의 메일은 절대 열어보지 마시고 즉시 목록에서 삭제해야 합니다.
2. 현재 폴더를 공유해 놓으신 분은 모두 공유를 모두 해제하시기 바랍니다.
(공유폴더를 통해 유입됨)
3. 문화재내부마당에 치료화일과 방법을 참고하시어 예방조치 하시기 바랍니다.

<감염시 조치>

1. 최신백신프로그램으로 바이러스 검사(감염화일삭제)
2. 시작- 찾기- 파일또는폴더 - *.eml - 지금찾기 - 모든화일삭제 - 재부팅

× "현재 폴더를 공유해 놓으신 분은
모두 공유를 해제하시기 바랍니다"

이 공지에 대응하는
"이제 다시 켜셔도 됩니다" 는 25년째 없습니다

작성자 「전산2」 · 조회 10,557

09.18. 발생 → 09.19. 언론 보도
→ 9. 20. 공지. 반응 속도는 이틀

오늘 제안의 결론

규제를 풀자는 게 아닙니다.
금지가 공짜인 구조를 바꾸자는 겁니다.

더 이른 학술정보 공유 이야기 - 유즈넷^{Usenet}

- 1981년 시작된 뉴스 서비스 웹(WWW)보다 10년 먼저 시작
- `han.comp.lang.c` 한국어.컴퓨터.언어.C 처럼 계층적으로 이름을 붙인 그룹에 게시
- 사용자가 가까운 위치의 뉴스서버에 게시하면
피어 서버를 통해 전세계 모든 뉴스서버가 동기화 되며 뉴스가 전달되는 구조
- 메일링 리스트와 영향을 주고 받으며, 웹 포럼 BBS · 레딧 · Discourse 으로 이어진 계보

중앙 서버도, 운영 회사도, 가입 심사도 없는 탈중앙화 구조

미국에서 쓴 글이 한국에 오는 길 (1994년 — 국경을 넘는 글은 전부 이 길로)



서상용, 「세계 수준의 연구전산망 뉴스 및 FTP서버 구축 관리」, 『지식정보인프라』 통권 4호, KISTI, 2001. 1. 15., 36쪽
KAIST 유즈넷 news.kaist.ac.kr 은 KAIST 학부 동아리 SPARCS 가 1992.12. ~ 2007.12. 운영

국가과학기술연구망 ^{KREONET} 이란

- **국내 과학기술연구 진흥을 위한 네트워크**
- 한국과학기술정보연구원 ^{KISTI} 이 운영
- 1988 국내 최초 전국 단위 TCP/IP 기반 인터넷 서비스 시작 ^{상용 보다 먼저}
- 2026년 서울-대전 1.2Tbps 백본으로 국내외 200여 산·학·연 기관 연동, 국제 400Gbps 백본으로 해외 연구망과 연동

누가 구축했나

연도	인물	역할
		현존 han 유즈넷 아카이브의 가장 오래된 게시물
1992. 12.	??? · KAIST	From: daemon@kaist.ac.kr Subject: 1992.12.01 han-kyoreh Sender: sysadm@worak.kaist.ac.kr Date: Tue, 1 Dec 92 01:18:54 GMT
1994	전밀반(John Milburn) · 포항공대 가속기연구소 초빙연구원	국내 뉴스서버망 구축 주도
1994	연구망	news.kreonet.re.kr 가동 — 하나망과 함께 국내 양대 백본
1995	한병길 · 구 시스템공학연구소	SPARC 1000으로 usenet.seri.re.kr 구축
1996~97	김재우 · 이승화 · 연구망	장비 보강 + PPRO-200 2대 추가 → 뉴스서버 4대 , 국내 최대 사이트
~2001	서상용 · 한국자원연구소	news.kreonet.re.kr 관리자 · INN 한국판 배포

서상용, 「세계 수준의 연구전산망 뉴스 및 FTP서버 구축 관리」, 『지식정보인프라』 통권 4호, KISTI, 2001. 1. 15., 36-37쪽
[Internet Archive - usenet-han 아카이브](#) 의 han.news 그룹에서 가장 오래된 뉴스넷 게시물 포함 확인
[1998-04-28, Google Groups - han.net.kreonet](#), [news.kreonet.re.kr 서비스 장애](#)

이분들 본업이 뭐였냐면

i	인물	연구소	본업
	전밀반	포항공대 가속기연구소	입자 가속기 연구
	서상용	한국자원연구소 석유해저자원연구부	석유·해저자원 연구

- 전산이 전공이 아닙니다. 뉴스서버 운영은 **누구의 KPI에도 없었습니다**
- 팀 버너스리 ^{웹(WWW)의 창시자} 도 같습니다 — **승인된 프로젝트가 아니었습니다.**
1989년 제안서에 그룹장이 적어 준 말은 **"Vague, but exciting"** 한 줄이었습니다

자기가 불편해서 만든 겁니다. 본업이 아니어서 만들 수 있었던 거고요.
아까 그 정의, 그대로입니다. **승인도 인지도 감독도 없었습니다.**

서상용, 「세계 수준의 연구전산망 뉴스 및 FTP서버 구축 관리」, 『지식정보인프라』 통권 4호, KISTI, 2001. 1. 15., 36-37쪽

Tim Berners-Lee, 「Information Management: A Proposal」, CERN, 1989. 3. — w3.org/History/1989/proposal.html · 그룹장 Mike Sendall 의 「Vague, but exciting」 메모

2005년, 유네스코가 "보존하자"고 지목했습니다

i 정보트러스트 어워드 2005 — 우리가 보존해야 할 인터넷
최초의 유즈넷이자 국내 IT기술창구 역할을 했던 KAIST 유즈넷 (news.kaist.ac.kr)



그 서버는 지금 없습니다. 어워드 2년뒤 사라짐



정보트러스트 어워드 2005 — 우리가 보존해야 할 인터넷
Internet Archive - news.kaist.ac.kr

필요했던 건 금지가 아니라 넛지^{Nudge}였습니다

아까 그 공지로 돌아갑니다

- 공유 폴더는 **일하라고 쓰던 것입니다** — 읽기 전용은 여럿이 참고하려고, 쓰기는 "주간업무보고 올리세요"를 각자 공용 파일을 직접 고치는 방식으로. 90년대 협업툴 없이 협업하던 방법입니다
- 많이 쓴다는 건 **그만큼 필요하다는 신호**입니다.
신호를 봤으면 할 일은 하나 — **같은 일을 더 싸고 더 안전하게 하는 방법**^{적정기술}을 찾는 것

✓ **평시 기본값은 바꿀 수 있었습니다**
읽기 전용을 기본값으로 · 쓰기가 필요한 폴더는 **하나만 따로**

i **사고 당일 끄는 건 맞습니다. 빠진 건 언제 다시 켜는가였습니다**

그런데 실제로 나온 선택지는 둘뿐이었습니다 — 공유를 끈다 / 제대로 한다

그래서 금지가 이겼습니다

	공유를 끈다	제대로 구축한다
비용	0원	예산 확보 · 사업 발주
시간	5분	수개월
결재	불필요	필요
인력	0명	운영 인력 상시
사고 시	책임 없음	담당자 문책

같은 월이 **메일로도** 퍼졌습니다. 그런데 메일은 안 꺾습니다 — 끄면 업무가 멈추니까

⚠ 0원짜리 금지와 수 억짜리 정답 사이에, 아무것도 없었습니다.
실질적인 선택지는 하나뿐이었습니다.

잘못된 건 구현 하나인데, 폐기된 건 개념 전체였습니다

감사의 비대칭

감사는 "왜 막지 않았나"만 묻고
"왜 못 쓰게 했나"는 묻지 않습니다

ftp.kreonet.re.kr 과 KAIST FTP — 같은 시기, 갈린 결말



시기	ftp.kreonet.re.kr	KAIST FTP
1990년대	1997. 3. 구축 · 미러 109종 · soc.culture.korean FAQ 에 자주 등장 · Slackware 한국 대표 미러 2곳 중 하나 · ftp.kr.kernel.org	cair-archive.kaist.ac.kr 한글 아카이브의 본거지 (초기 오픈소스 소프트웨어 한글화 노력)
90년대 후반	—	사라짐 밀줄 ①
2002	—	학부생 개인의 하드웨어로 서버를 재건
2010	사라짐 밀줄 ②	장비 노후화로 잦은 장애 → 신규 하드웨어 교체
2014	—	하루 3,500명 · 매일 100만 건 · 1TB 전송 · 60종 13TB
2024	—	KROENET 100Gbps 회선 지원
2026	—	kr.archive.ubuntu.com 외 — 그런데 후임이 잘 안 뵈힙니다

i 한쪽은 죽었고, 한쪽은 죽었다가 살아났습니다. 차이는 예산이 아니라 사람 한 명이었습니다.

2014. 7. 31. 카이스트신문 독자기고 KAIST FTP의 어제와 오늘, 그리고 앞으로
Internet Archive - ftp.kreonet.re.kr

팀 단위로 필요한 새도우 IT

외부 서비스인지 직접 설치인지는 구분하지 않겠습니다. 필요한 기능만 적겠습니다

기능	무엇을 하려고
위키	회의록 · 업무정보 · 인수인계 기록
메신저	업무 연락 — 폰에서 바로 받아야 함
캘린더	폰으로 동료 휴가 · 출장 현황 파악
모니터링	실험장비 구동현황 파악
공동 논문작성	외부자 포함 실시간 동기화 + 코멘트
코드 · 데이터	분석코드와 데이터를 같이 두고 재현
분석 환경	환경을 내 맘대로 바꿀 수 있는 GPU
AI 도구	코드 · 문서 작업 보조

? 이 중에 기관이 주는 게 몇 개입니까?

없거나 · 느리거나 · 내 맘대로 못 하거나 · 행정절차가 많거나

일하는 사람에게는 전부 같은 뜻입니다 — 지금은 못 씁니다

일을 하려면 필요한데요?

고립이 면죄부입니다

실험장비에서 데이터를 빼내려면 옆에 **ADC · 라즈베리파이**를 붙입니다. 그 순간 —

보안성 검토	
제조사 완제품, 네트워크 미연결	대상 아님 — 기납품이라 변경 불가 그 안에서 20년 된 WinXP 가 돌고 있어도
거기에 계측 게이트웨이를 붙임	비인가 외부 장치 → 폴스캔 기준이 매년 바뀌어 작년 소명서를 못 씁니다
그 게이트웨이를 검사하려면	스캐너 · 에이전트가 x86 서버용입니다

⚠

계측 게이트웨이는 **ARM · RISC-V** 입니다.
검사를 돌리겠다고 라이브러리를 욕여넣으면 — 임베디드의 장점인 최소 공격면을 스스로 버립니다

데이터를 흐르게 하려는 행위가 검토를 부르고, 검토를 통과하려는 행위가 공격면을 늘립니다

2026-06-24, 장민석, 연구자가 메타데이터를 입력하지 않게 하고 싶습니다. ESR(Electronic Science Record) 생태계 제안
2026-06-24, 장민석, 연구데이터는 왜 흐르지 않는가 — 그리고 그 해법(ESR)은 어떻게 말할 수 있는가

그리고 AI는 이미 OS 와 웹브라우저 안에 들어왔습니다

- **Microsoft Recall** — 화면을 주기적으로 찍어두고 AI가 검색·요약해줍니다
- 스마트폰에 대고 말하면 **LLM 이 앱을 직접 제어**합니다 — 일정 추가, 알람 설정, 메시지 발송
- **브라우저에도 들어왔습니다** — 크롬 **Gemini**, 엣지 **Copilot**. 지금 열어 둔 **그 페이지**를 읽고 요약합니다
- 앱 하나를 막는 게 아니라 **운영체제와 브라우저에 들어와 있습니다**
- 금지하면 안 쓰는 게 아니라, **더 편한 바깥** 것을 씁니다
- 기관이 통제할 수 있는 건 줄고, 새도우 IT는 **늘어납니다**

금지의 대상이 앱에서 **OS 와 브라우저**로 옮겨왔습니다. 그 요구사항 목록은 앞으로 더 길어집니다.

이걸 금지하실 겁니까?

막으려면 기준이 있습니다 — SCIF

Sensitive Compartmented Information Facility

극단적으로 막는 방법

- 연구소 입출입 **소지품 엑스레이 검사** - 수첩까지 걸러내기
- 제한구역 접근시 **폰/노트북 사물함**에 보관
- **전파가 안 닿는 구역**에서 업무수행

ICD 705 미국 정보기관 민감정보 취급 시설 기준

요건	내용
개인 전자기기	반입 금지 — 정부 승인·차폐된 것만 예외
전파 차폐	≥ 40 dB, 10 kHz ~ 10 GHz
전자파 방출	TEMPEST 대책 — 지정 기관(CTTA) 평가
건축	신축 때 반영. 사후 적용은 비용이 훨씬 큼니다

- i

국내에도 **보호구역 3단계**가 있습니다 — 제한지역 · 제한구역 · **통제구역**(비인가자 출입 금지).
다만 **전자기기 반입 통제**는 그 규정에 없습니다. 기관 시행세칙 사항입니다
- ?

남의 나라 얘기가 아닙니다. 저도 겪었습니다 — 10년 전 어느 국가 연구기관 면접.
폰은 경비실에 맡기고, 금속탐지기로 몸을 스캔하고, 소지품 검사를 받고 들어갔습니다. **방문객에게도** 그렇게 합니다

수첩 한 권과 폰 한 대가 들어오는 환경이면, 통제 수준은 이미 정해진 겁니다

ICD/ICS 705 Technical Specification · PED 반입 금지 · RF 차폐 ≥40 dB (10 kHz~10 GHz) · TEMPEST
보안업무규정 제60조 — 보호구역

가장 강하게 나는 곳에서도

망분리 기관이 있습니다 — 내부망과 외부망을 아예 따로 둡니다

	내부망	외부망
데이터	반출 금지 대상	공개 가능한 시뮬레이터 · 형상
연구	소 안에서만	노트북 반출 · 웹메일 가능
합치는 길	← 망간접점 시스템으로 밀어넣기	

i

노트북이 나가고 웹메일이 되는 건 통제가 느슨해서가 아닙니다.
외부망 기준이라 그렇습니다. — 밖에 있는 건 이미 밖에 있어도 된다고 분류를 통과한 것입니다

⚠

그런데 개인 단말은 여기서도 들어옵니다.
망분리가 막는 건 내부망 연결 이지 반입 이 아닙니다. 내부망에서 작업하면서 개인 폰으로 전화 받습니다

?

그리고 연구는 칼로 두부 자르듯 나뉘지 않습니다.
아르키메데스는 목욕탕에서 풀었습니다. 요즘 연구자는 사비를 들여 GPU 데탑을 집에 마련하고,
Claude Code 을 원격으로 걸어두고, 화장실에서도 피드백을 받습니다. 그리고 그 결과를 들고 월요일에 들어옵니다

밖에서 하는 건 일탈이 아니라 연구 연속성입니다. 막은 건 데이터지 머리가 아닙니다

그래서 지금 우리가 만들 수 있는 것

주소 없음 (서버 X, 개인 PC 구동)		주소 있음
공개 데이터	로컬 도구	GitHub Pages · Vercel
내부 데이터	깃헙으로 배포, 승인없이 전국으로 확산	공공 AI/AX 사례 대부분
		조직 자산이 되는 앱 - 사례 없음

✓ **KorDoc** - HWP · HWPX · PDF 를 마크다운으로 바꾸는 로컬 도구, 서버가 없으니 **승인도 필요 없습니다**

i **판단 없이 혼자 할 수 있는 건 왼쪽만**
PC 안에서 외부 통신 없이 동작하거나, 공개 API 만 쓰는 것
이 외에는 **서버를 띄워 남에게 보여주는 순간 신고 대상**이 됩니다

그런데 앞의 요구사항은 전부 **「함께 보고 협업하는 것」**입니다 — 위키 · 캘린더 · 모니터링 · 공동 논문작성.
도구는 퍼뜨릴 수 있는데, 같이 활용하는 것을 못 만듭니다

2부 · 비전공자가 하기 쉬운 실수, 그리고 안전하게 운영하는 법

0:12 - 0:18

막는 게 아니라 안전하게 하는 법.
실제로 본 실수 열 가지와 그 대응.

진짜 위험이 뭔지부터 — 지형이 바뀌었습니다

	예전 인식	지금 실제
랜섬웨어	제일 무섭다	공격은 기록적으로 늘었는데 돈이 안 됩니다 — 지불액 2024년 35% 감소, \$813M . 피해자가 안 냅니다
개인정보 유출	최우선	돈이 잘 안 됩니다 (다크웹 민번 1건당 10원 꼴)
채굴	잘 모름	자원만 있으면 즉시 현금. 협상도 피해자도 필요 없습니다
공급망	잘 모름	연구소에서 제일 위험. 앱이 정상 동작하는 채로 데이터가 나갑니다

i 랜섬웨어는 돈이 안 되기 시작했고, 채굴은 바로 돈이 됩니다

그래서 연구소에서 무서운 건 둘입니다

- "오픈소스 코드를 타고 내부를 공격한다"는 일은 사실상 없습니다
- 실제로 벌어지는 건 — 방화벽이 뚫리거나 API가 부정 이용돼 내부 GPU로 채굴
→ 그리고 이건 방화벽으로 잘 막힙니다
- 진짜 무서운 건 공급망 공격으로 데이터가 나가는 것
→ 이건 방화벽의 나가는 쪽을 막아야 합니다

들어오는 걸 막는 얘기는 많이 들으셨을 겁니다.
오늘은 나가는 쪽 얘기를 하겠습니다.

하기 쉬운 실수, 그리고 안전하게 운영하는 법

	하기 쉬운 실수	문제가 되는 것	안전하게 운영하는 법
방화벽 In	01 도커 올리고 ufw 로 막은 줄 안다	포트 열림 → 침입 → GPU 채굴	<code>ufw-docker</code> 또는 <code>DOCKER-USER</code> DROP
	02 들어오는 쪽만 막고 나가는 쪽은 그대로	공급망 → 데이터 유출	호스트 <code>ufw default deny outgoing</code> · WAS·DB 는 <code>compose internal: true</code>
	03 그럼 패키지는 어떻게 받나	CDN 이라 허용목록이 사실상 인터넷 열기 → 02 가 무력화	squid 한 대로 허용목록을 내부 IP 하나로 · 진지하면 pull-through 캐시
	04 인증서 갱신 때문에 다 열어둔다	LE 검증 출발지를 못 좁혀 80/443 전면 개방	DNS-01 — 인바운드가 아예 불필요 · <code>_acme-challenge</code> 만 위임
접근 통제	05 로그인을 직접 만든다	자작 인증 취약점	IdP + <code>forward_auth</code> — 앱은 로그인을 구현하지 않음
	06 앱이 늘수록 계정이 흩어진다	퇴사자 계정 잔존, 권한 추적 불가	IdP 하나로 공통 관리
	07 앱이 그대로 노출된다	프레임워크 지문 · 버전 배너 · 스캔	mTLS — 없으면 TLS 단계에서 끝 부담되면 → authentik outpost
운영 격리	08 서버를 손으로 고친다	재현 불가 · 복구 불가	Git이 정답 — <code>compose</code> 로 재생성
	09 로그가 그 기계에만 있다	침해되면 증거가 없다	다른 기계로 전송
	10 하이퍼바이저를 안 만진다	게스트 침해 시 방어선 없음	Proxmox 방화벽 = VM 밖

앱은 무한히 늘어도 상자는 하나입니다

01 도커를 올렸는데 ufw로 막은 줄 압니다

! `ufw default deny incoming` 을 걸어 두고 막혔다고 생각합니다.
컨테이너에는 안 걸립니다 — Docker 가 iptables 를 직접 조작하고,
컨테이너 트래픽은 `INPUT` 이 아니라 `FORWARD` 체인으로 갑니다

× 포트는 그대로 열려 있습니다 → 침입 → 내부 GPU 채굴

✓ `ufw-docker` · 또는
`iptables -I DOCKER-USER -j DROP` 뒤에 필요한 것만 **ACCEPT**

02 나가는 쪽은 아무도 안 막습니다

! 들어오는 쪽만 막고 나가는 쪽은 그대로 둡니다.

그런데 공급망 유출은 빌드 시점이 아니라 런타임 상시 연결에서 일어납니다 → 공급망 → 데이터 유출

층	막히는 것	수단
호스트 프로세스	네이티브 앱의 아웃바운드	<code>ufw default deny outgoing</code> 웹 안 쓰는 노드(DB · 계산 · 워커)는 거의 공짜
바깥과 맞닿은 컨테이너	나가야 하니 목적지만 좁힘	<code>DOCKER-USER</code> 체인 · 지정 프록시만
WAS · DB	아웃바운드 전면	compose <code>internal: true</code>

⚠ 호스트의 `deny outgoing` 은 컨테이너에 안 걸립니다. 컨테이너 트래픽은 `OUTPUT` 이 아니라 `FORWARD` 로 갑니다 — 01 의 그 함정이고, 방향만 반대입니다

제일 깊은 격리는 방화벽 규칙이 아니라 compose 한 줄입니다

02 그 한 줄 — internal: true

```
networks:
  edge: { driver: bridge }      # 바깥과 맞닿는 유일한 망
  app:  { internal: true }     # ← 이 한 줄이 아웃바운드를 끊습니다
  data: { internal: true }

services:
  proxy: { networks: [edge, app], ports: ["443:443"] }
  was:   { networks: [app, data] }   # edge 에 없음 → 나갈 길이 없음
  db:    { networks: [data] }        # 가장 깊은 자리
```

⚠ **edge** 에 같이 붙이는 순간 무효입니다. 망 두 개에 붙으면 나갈 수 있는 쪽으로 나갑니다

✓ 가끔은 나가야 합니다. 그때는 타이머를 같이 겁니다

```
systemd-run --on-active=15min --unit=net-close /usr/local/sbin/net-close
```

수동 토글이 아니라, 만료되는 예외입니다

03 그럼 apt · pip · conda는 어떻게 하죠?

! 02 를 걸어도 패키지는 받아야 합니다. 그래서 허용목록을 엽니다
— 그런데 **PyPI·npm** 은 **CDN** 이라 호스트가 계속 바뀝니다. 도메인으로도 IP 로도 못 줍니다

× 허용목록이 사실상 「인터넷 열기」가 됩니다 — **02 가 무력화됩니다**

☐ 제대로 된 답은 기관이 미러를 하나 운영해 주는 것입니다.
그러면 팀마다 따로 세울 일이 없습니다. 우리는 없습니다

✓ 그래서 **squid 한 대**. 컨테이너는 squid 만 보고, 나갈 수 있는 건 squid 뿐 — 허용목록이 **내부 IP 하나**가 됩니다. 진지하게는 **pull-through 캐시**(Harbor · devpi/bandersnatch)

허용목록이 아니라 단일 오리진이 되고, 재현성·감사 로그가 덤입니다

04 인증서 갱신 때문에 다 얼어듭니다

! 예전엔 회계연도마다 한 번 사서 갱신하면 끝이었습니다.
수명이 **200일** → **100일** → **47일**로 줄고 서비스까지 늘면 사람 손으로는 못 합니다 → LE(Let's Encrypt)
그런데 **HTTP-01** 은 **LE** 가 어디서 검증하러 올지 모릅니다
— 출발지 IP 를 일부러 공개하지 않고, 여러 지점에서 동시에 확인합니다. **ACL** 에 넣을 대상이 없습니다

✕ 그래서 **80/443** 을 전 세계에 열어 둡니다 — 공인 IP 에 **ACL** 로 좁혀 두려던 게 도로아미타불

✓ **DNS-01** 은 인바운드가 아예 필요 없습니다. TXT 레코드만 쓰면 되니 **ACL** 을 그대로 둡니다.
와일드카드도 DNS-01 에서만 나옵니다. compose 안에서는 **caddy**만 아웃바운드 허용

i 대신 **DNS** 쪽 부담이 생깁니다 — 그런데 **_acme-challenge** 만 위임받으면 됩니다.
존 전체를 맡지도, 부서가 DNS 서버를 운영하지도 않습니다. 토큰 권한도 **그 레코드 하나로** 좁힙니다

DNS-01 + 와일드카드 — 앱이 늘어도 발급 이벤트는 안 늘어납니다

05 로그인을 직접 만듭니다

! 자작 로그인이 세상에서 제일 위험합니다

✗ 자작 인증 취약점 · 그리고 헤더를 앱이 믿는 구조면 내부망에서 **헤더 하나로 아무나** 됩니다

✓ Caddy `forward_auth` 를 **공통 include** 로
— 로그인 전엔 앱이 응답조차 안 합니다.
앱 코드도, 버전 배너도, 프레임워크 지문도 노출되지 않습니다.
프록시가 넘기는 신원 헤더는 반드시 **strip** 후 재삽입

06 앱이 늘수록 계정이 흩어집니다

! 새도우 IT 하는 사람은 **하나만 안 합니다**. 좋은 게 보이면 하나씩 더 붙입니다.
바이브코딩으로 앱도 만들고요 — **앱이 늘 때마다 계정이 가장 큰 문제가 됩니다**

✕ 퇴사자 계정 잔존 · 권한 추적 불가

✓
기관 IdP
↓ (등록 1건)
iam.부서.example.kr – authentik (기관엔 SP, 앱엔 IdP)
↓
부서 앱 · 앱 · 앱 · 앱 ...

- 기관 IAM 담당자 입장에서 등록 요청이 **100건 → 1건**
- 앱들은 기관 IdP 속성 전체를 못 받고 **브로커가 거른 것만** 받습니다
- DNS 는 `_acme-challenge` 만 위임받으면 **부서가 운영할 서버가 없습니다**

통제 완화가 아니라 통제 대상 축소입니다

07 앱이 그대로 노출됩니다 — 노출 면 줄이기

✕ 프레임워크 지문 · 버전 배너 · 스캔 — 앱이 그대로 보입니다

목적은 하나입니다. 앱을 세상에 안 보이게 하는 것. 방법이 둘이고 강도가 다릅니다.

	막히는 지점	공격자가 알아내는 것	부담
mTLS	TLS 핸드셰이크	아무것도. HTTP 응답이 아예 없음	기기마다 인증서 설치·갱신
authentik outpost	HTTP 요청 (<code>forward_auth</code>)	프록시가 있다 · authentik 을 쓴다	없음

- ✓ **mTLS 가 훨씬 강합니다** — 앱은커녕 프록시 응답도 못 봅니다.
그런데 인증서 까는 걸 다들 귀찮아해서 현실에서는 outpost 를 쓰게 됩니다.
`ez-mtls-manager` 는 바로 그 부담을 줄이려고 만들었습니다 — 자가발급 + 다중 IdP 인증 + 갱신 자동화

둘 중 뭘 쓰든, 앱 개발자는 아무것도 안 해도 됩니다

08 서버를 손으로 고칩니다

✕ 재현 불가 · 복구 불가

✓ `compose.yaml` 과 `Caddyfile` 이 저장소에, 서버는 거기서 생성됩니다.
손으로 고친 건 다음 배포에 사라집니다 — 그게 정상입니다

서버 날리고 30분 안에 복구되나?
안 되면 그건 서버가 아니라 애완동물입니다.
볼륨 백업까지 포함해서 세야 합니다

09 로그가 그 기계에만 있습니다

✕ 침해된 기계 위의 로그는 증거가 아닙니다

✓ 다른 기계로 보냅니다. 제일 값진 로그는 성공이 아니라 **차단된 아웃바운드 시도**
— 공급망 공격이 실제로 일어났다는 **유일한 증거**입니다

감사에서 "근거 있습니까" 에 답할 유일한 자료입니다

10 하이퍼바이저를 안 만집니다

! 방화벽을 게스트 안에만 겁니다

× 게스트가 침해되면 방어선이 같이 넘어갑니다

✓ Proxmox 방화벽은 VM 바깥에 있습니다. 게스트 안에서 root 를 먹어도 못 뚫니다

- 정책이 적용된 VM 템플릿으로 찍어내면 그게 곧 "클론하면 정책 승계"
- 계정·암호 분실? cloud-init 설정 갱신 후 부팅하면 새 정보로 접속
- SSH 는 관리 대역만 + 키 전용 — noVNC 콘솔이 있으니 잠길 일이 없습니다

오래된 연구SW가 신규 OS를 지원 안하는 경우 - 못 고치면 감쌉니다

3부 · SADP Secure App Deploy Profile

0:18 – 0:25

열 개 보셨죠. 매번 하실 겁니까?

신경 안 쓰게 하면서도, 안전하게

! **needs** 는 있습니다. 그런데 실수하기 쉽고, 고려할 게 너무 많습니다

- 바깥 서비스는 막혀 있고, 그래서 **협업까지** 같이 막힙니다
- 오픈소스는 **더** 들여와야 하는데 들여올 길이 좁습니다
- 바이브코딩 앱은 늘어나는데 — 만든 본인도 코드를 다 설명하지 못합니다

× 그렇다고 **하나하나 검토 받으라고** 하면 **학을 땀니다.**
안 하거나, 숨기거나 둘 중 하나입니다

금지도 아니고, 수억짜리 사업도 아닌 것 — **적정기술** 이 필요합니다.

해외는 어떤 노력을 하나요? — Trusted CI

- 미국 **NSF 사이버보안 우수센터** NSF Cybersecurity Center of Excellence **수행**
- 2012년부터 **14년째 진행중** NSF 1234408 · 1547272 · 1920430 · 2241313 **(연 600만 달러 ^{약 80억} 규모)**
- 연구 인프라 및 관련 오픈소스 프로젝트에 **1:1 로 붙어서** 보안을 강화

대상	무엇을 했나	
Jupyter	2021	보안 문서가 어디에 비었는지 전수 조사 + 배포 유형별(슈퍼컴센터 · 캠퍼스) 모범사례 작성
Tapis	2024	코드 수준 보안 점검 — 심각한 취약점 4건 + 버그 1건 발견, 개발팀이 반영
하와이 Gemini 천문대	2015-16	데이터 무결성 확보 보안 로드맵 + 우선순위 평가지표(Water line) 도입 · 사용자 보안 교육
시카고 Array of Things	2016	도시 센서망의 보안 · 개인정보 보호 정책 검토 + 대중 소통 · 투명성 확보 지원

무엇을 중요시 하나요? — Trusted CI

“ **Framework**

기존의 단편적 통제 지침에서 벗어나,
조직 고유의 미션에 최적화된 방식으로 사이버 보안을 지속 가능하고 체계적으로 실행하도록 돕는 프레임워크

“ **OSCRP** — Open Science Cyber Risk Profile

공격자나 수법이 아니라, **자원을 잃으면 무엇을 못 하게 되는가**로 기술한 문서

Asset 자산	Concern 무슨 일이	Consequence 그래서 무엇을 잃나
Documentation (wiki)	Inaccessible	Lost science time
Network-connected sensor control system	Inaccessible network (flooding attack)	Lost science time, affecting unrelated projects at the same facility
Internal data	Data exposed	Reduced reputation · legal action

과학자들(**working group**)이 만들어 올렸습니다. 위에서 내려온 게 아닙니다 — 누구나 **GitHub** 으로 문제를 제기합니다

개방을 깨지 않는 보안은 이미 하나의 분야입니다. 우리는 그 가이드가 없습니다

Framework Core — 4 Pillars · 16 Musts · [OSCRP 전문](#) · [저장소\(이슈로 의견 접수\)](#) · [IEEE S&P 2017](#) — 2017년 문서, 공개 저장소 최종 갱신 2021

[Success Stories](#) · [웹비나 2016~2025](#) · [YouTube](#) — 2023. 1. **RTEMS · EPICS** 보안 · [arXiv 1904.05383 \(PEARC '19\)](#) · 2024 갱신 연 600만 달러 · 2029. 9.까지

우리는 어디까지 약속해야 할까요?

과장된 약속

침입을 막습니다

탈취물이 못 나갑니다

앱이 안전합니다

전부 추적됩니다

검증 가능한 약속

공격자가 목적지를 임의로 고를 수 없습니다

앱마다 데이터 접근권한을 줍습니다

침해된 앱을 격리하고 복구할 수 있습니다

접근 로그는 항상 남고 다른 기계로 보냅니다.

OIDC 를 거친 경로에는 거기에 누가까지 붙습니다

```
caddy log_append auth_sub {http.request.header.X-Auth-Request-User}
```

우리는 무엇을 제어할 수 있나요? — 4개 통제 지점

통제 지점	안 해도 되는 것	플랫폼이 대신 하는 일
입구	01 05 06 07	조직 IP 대역만 + 인증 프록시(SSO) + HTTPS 자동
출구	02 04	컨테이너 아웃바운드 전면 차단, 지정 프록시만
빌드	03	허용된 프록시 경유로만 패키지 수급, 고정 태그·digest 만
기록	08 09	접근·배포·변경 전부 자동 기록. 정답은 Git, 임의 수정 시 자동 복구

앱은 `my_app.my_team.my_company.net` 에 생성됩니다

2부는 열 개를 외우라는 얘기였습니다.
이건 그걸 각자 안 해도 되게 하는 얘기입니다

쓰는 분이 알아야 할 건 한 문장입니다

✓ 「내부 Git 의 앱 베이스를 클론해서 만드세요」

앱 만드는 사람이 하는 일

- 1 목적에 맞는 앱 베이스를 고른다 — 목록에서
- 2 `git clone`
- 3 자기 기능만 만든다 — 로그인 · HTTPS · 접근제어는 손대지 않는다
- 4 `git push` → 자동 배포. `my_app.my_team.my_company.net` 에 뜬다
- 5 동료에게 주소만 보낸다. 로그인은 기관 계정 그대로

미리 지정된 통제 지점을, 쓰시는 분은 고르고, 클론하고, 올리면 됩니다

관리자가 한 번 해두는 일

목적별로 빈 앱 베이스를 만들어 둡니다. 프리셋이 박힌 채로

베이스에 박아 두는 것		
누가 보나 ^{ID · 그룹}	① 팀 · ② 전 직원 · ③ ANY	3
어디서 보나 ^{IP}	① 팀 · ② 회사 · ③ 국내 · ④ 전 세계	4
OIDC 프록시	① 없음 · ② 활성화 사용자별 URL 요청이 로깅되고, 로그인 전까지 앱 코드가 노출되지 않습니다	2
무엇과 붙나	① 없음 · ② 내부 API 제공자 A	2
경우의 수		48

48 가지가 나오지만, 실제로 쓰는 건 서너 개입니다. 그 서너 개만 베이스로 만들어 둡니다
실험 대시보드를 부서 안에서만 보려면 — ①①①①

앱마다 규칙을 만들면 앱 수만큼 늘지만, 베이스는 패턴 수만큼만 늘니다

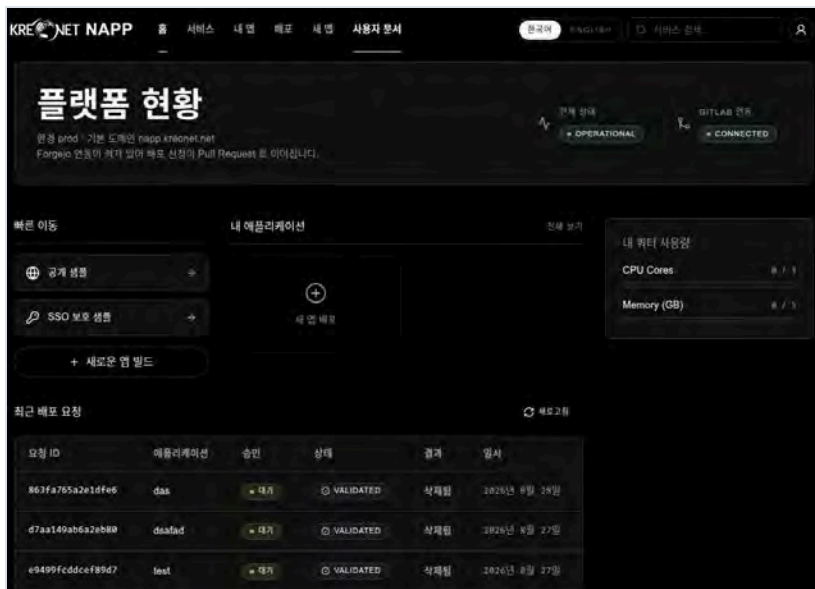
그래서 상자 위에 뭘 올립니까

! 못 쓰게는 하는데 대체가 없습니다. 그러면 밖으로 나갑니다

계열	지금 개인이 쓰는 것	팀이 올릴 수 있는 것
위키	Notion	Outline
메신저	Slack	Mattermost — 모니터링·봇 통합
코드 관리	GitHub	GitLab · Gitea · Forgejo
메일링	Google Groups	Sympa · HyperKitty
이벤트	—	Indico
내부 자료 쓰는 앱	—	바이브코딩으로 만든 그것

1부에서 「같이 활용하는 것을 못 만든다」고 했던, 그 칸이 이것입니다

두 구현 — 같은 약속, 다른 구현



- 기관 규모: RKE2 · ArgoCD
- 팀 규모: docker compose · Coolify

✓ 둘 다 같은 네 가지를 지킵니다
— 입구 · 출구 · 빌드 · 기록.
밑바닥이 달라도 적합성은 같습니다

"침입을 막았다"는 증명되지 않지만, "탈출이 막혔다"는 증명됩니다

4부 · 손잡이

0:25 - 0:32

기술은 됩니다. 그런데 만들어도 안 쌓입니다.
불을 데가 없어서요.

AI 는 손잡이로 일합니다

- 클로드·GPT 가 잘 하는 건 모델만이 아닙니다. **도구를 정확히 부르는 것(tool calling)**입니다
- 손잡이가 없으면? **AI 가 프론트·백엔드를 훑어서 어떻게든 짭니다.** 그리고 개편 한 번에 깨집니다

제 기관 AI 활용 경진대회 1차 평가 — 123명이 뽑은 상위 넷

	제안	표
1	□□□□□□ — 업무보조 AI 에이전트	61
2	□□□□□□ — 규정·지침 특화 챗봇	34
3	LLM 산출물을 HWP 로	29
4	RAG 문서검색 + MCP 연동	23

넷 다 모델이 아니라 통로입니다. 조직이 이미 그렇게 투표했습니다

입사일 이전 메일이 검색되시나요?

가장 오래된 손잡이 — 메일링 리스트와 아카이브

저는 2013년에 들어간 회사에서, 입사 전 우리 팀 메일이 전부 검색됐습니다

- 팀 메일주소로 오간 것이면 내가 오기 전 것도 보입니다
- 다른 팀에 뭘 부탁하는 메일에도 우리 팀 주소를 넣습니다. 선임자도 그렇게 했고요
- 그래서 신입도 알 수 있었습니다 — 저 팀엔 누구를 통해, 어떤 순서로, 어떤 어투로 요청하는지
- To 에 팀메일을 안 넣으면 혼났습니다.

이유는 보안이 아니라 "나중에 다른 사람이 이 일을 못 이어받아서"

인수인계 문서는 결론을 알려주지만, 스레드는 일하는 방식을 알려줍니다

i 사기업은 사람이 나가는 것을 전제로 기록을 설계합니다

출연연에 와서 가장 놀란 것

스스로의 역사를 버립니다 역사: 미시사 微視史 Microhistory

- 팀 주소는 있습니다. 없는 건 그걸 검색할 웹 아카이브입니다 — 보내고 끝입니다
- 메일 시스템에서 팀을 고르면 개인 주소로 치환되어 1:1로 N번 나가기도 합니다
- **shared mailbox**(내 메일함을 남과 나눠 보는 것) 개념도 없습니다

✕ 그래서 공지메일을 못 지웁니다

아카이브가 있으면 "나중에 찾으려면 되지" 하고 지웁니다.

없으니까 각자의 받은편지함이 곧 아카이브입니다

- ✓ 팀메일 + 웹 아카이브가 있으면 위키도 필요 없습니다
— 보내면 그게 기록이고, 정리할 사람도 필요 없습니다

여러분 받은편지함이 이 조직의 아카이브입니다. 그게 문제입니다

2026년, 같은 일이 벌어집니다

AX 를 한다는 어느 출연연 — 내부 GPU 로 돌리는 LLM 은 느립니다

✓ 그래서 메일로 요청하고 메일로 답을 받게 만들었습니다 — 좋은 설계입니다
느린 걸 비동기로 감싸는 건 정석이고, 사용자는 이미 메일을 씁니다 — 문턱 비용 0

! 그런데 그다음이 이렇게됩니다
To 에 팀원을 전부 넣습니다 · 봇 전용 메일 주소를 따로 만들어 거기도 넣습니다

팀 메일링과 웹 아카이브가 있었다면
— 주소 하나로 보내고, 나머지는 봇이 아카이브를 읽으면 끝입니다

없는 건 AI 가 아니라 주소 하나와 아카이브 하나입니다

손잡이가 없으면 쌓을 수 없습니다

- 기존 시스템에 뭘 이어 붙이려면 **손잡이**가 있어야 합니다.
그 손잡이가 **주소(permalink URI)**입니다
- 주소가 없으면 가리킬 수 없고, 가리킬 수 없으면 **인용도 연결도 자동화도 안 됩니다**
- 그래서 매번 처음부터 다시 만듭니다. 쌓이지 않고 **붕괴합니다**

근거

주소가 있어서

2001년 국가유산청 공지

25년 전 게시물인데 링크가 삽니다

서울시 결재문서

총량제 설명이 캡처 한 장으로 끝납니다

유즈넷 Message-ID

본문이 89% 깨져도 누가 언제 무엇에 답했는지는 남았습니다

오늘 이 발표가 근거를 댈 수 있었던 건, 전부 주소가 있었기 때문입니다

손잡이가 없으면 쌓을 수 없습니다

반대로 연구소의 결재문서·회의록·팀 메일은, 이 발표에 근거로 넣을 수 없었습니다. 가리킬 주소가 없습니다

× DOI 등록관리기관을 운영하는 제 기관 — 연혁 페이지 주소는 kisti.re.kr/intro/pageView/15
제 연구자 페이지는 화면에 ISNI 를 띄워 놓고, 정작 주소는 [?cn=ADPER6801759152](https://kisti.re.kr/intro/pageView/15?cn=ADPER6801759152) 입니다

! 있던 주소가 죽기도 합니다 — 2020년 9월, 질병관리본부 cdc.go.kr 가 질병관리청 kdca.go.kr 로
승격했습니다. 코로나19 한복판이었습니다
지금 옛 주소로 들어가면 인증서 경고가 먼저 뜨고, 넘겨도 루트로만 보냅니다 — 경로는 버려집니다

"바퀴를 다시 발명하지 말라"는 말은, 제대로 된 바퀴가 있을 때만 성립합니다.
우리는 제대로 된 바퀴를 가지고 있습니까?

출처 — IT동아 김대은 「질병관리청은 왜 '죽은 링크'를 관리하지 않을까」 2021. 4. 27. it.donga.com/31878 · 동아일보 재게재 106636945

보조 — 임민철 「정부의 웹사이트 총량제는 코로나19 방역을 어떻게 방해하나」 mincheol.im/143 (북저널리즘 2021 → 2025. 3. 공개 · 논증 위주, 폐쇄 사례 목록은 없음)

실측 2026. 9. 17. — `curl -sKI -L https://www.cdc.go.kr/kdca/5468/subview.do` → ① 인증서에 [cdc.go.kr](https://www.cdc.go.kr) 이름 없음 ② 302 Found → www.kdca.go.kr 경로 소실

ScienceON 연구자(장민석) — scienceon.kisti.re.kr/...?cn=ADPER6801759152 · 화면 표시 ISNI 0000 0004 6784 8612

ISNI KOREA — 국립중앙도서관 · KISTI 는 2016년 세계 10번째 DOI 등록관리기관(RA)

웹사이트 총량제

SADP 는 앱마다 도메인을 만듭니다 — `my_app.my_team.my_company.net`

- 큰 앱일 수도, 작은 앱일 수도 있습니다
- 보여줄 대상도, 열어줄 IP 범위도 제각각입니다
- 가져다 쓴 오픈소스일 수도, 바이브코딩으로 만든 것일 수도 있습니다

× 그런데 총량제는 도메인 단위입니다

팀 도메인 `my_team.my_company.net` 도, 앱 도메인 `my_app.my_team.my_company.net` 도 각각 대상입니다 — 개별로 신고하고 관리하고, 기관 전체 개수를 줄여야 합니다
근거는 행정안전부 「행정·공공기관 웹사이트 구축·운영 가이드」
— 기타 공공기관에서 해제된 출연연에도 내려옵니다

저 목록에 올라가는 순간 학을 뗍니다.

사람들이 괜히 공개 데이터 앱만 만들고, `github.io` 와 Vercel 을 쓰는 게 아닙니다

서울시 결재문서 — 「행정·공공기관 웹사이트 총량제 예외 증빙자료 제출」 · 기관별 총량이 배정되고, 신설하려면 예외 증빙자료를 냅니다
그리고 새 주소를 만들려면 기존 도메인을 반납해야 합니다 — 반납은 리다이렉트를 구조적으로 불가능하게 만듭니다

5부 · 정책 제안

0:32 - 0:40

바뀌야 할 건 기술이 아니라 가격표입니다.
작고 싼 것부터 아홉 가지.

모든 연구가 민감정보는 아닙니다

- 천체물리 관측 · 자연현상 · 기후 데이터
- **미사일도 핵시설도 아니고, 개인정보도 없습니다**
- 그런데 통제는 동일하게 걸립니다
- AI로 앱 만드는 건 쉬워졌는데,
나누려니까 배포·도메인·HTTPS·로그인·IP통제가 붙습니다

연구는 고이면 퇴화합니다

그런데 제도는 이미 바뀌었습니다

- **2026. 5. 1.** 「국가 사이버보안 기본지침」 시행 — 기존 **망분리 조항 공식 폐지**
- **N2SF** — 데이터를 등급으로 나눠 통제를 **차등 적용**

등급	유출되면
C Classified · 기밀	국가안보 · 기관에 심각한 피해
S Sensitive · 민감	업무 · 개인에게 피해
O Open · 공개	피해가 없거나, 어차피 공개될 것

제가 지금 성토한 게 정확히 N2SF가 고치려는 것입니다

그런데 과분류는 여전히 공짜입니다

- 차등 적용하라는데 판단 주체는 현장입니다
- 그런데 현장에는 판단 근거도, 판단했다가 보호받을 장치도 없습니다
- O^{공개} 로 분류했다 사고 나면 문책, 전부 C^{기밀} 로 찍으면 아무 일도 없음

× 합리적인 담당자의 최적 전략은 여전히 "전부 C"입니다

그리고 기준은 늘어나기만 합니다 — 보안 기준은 국정원, 감사 기준은 감사원 디지털감사국. 디지털감사국은 2024년에 생겼습니다. 현장은 늘 가장 엄한 쪽에 맞춥니다. 그게 합리적이니깐요.

기준을 합치자는 게 아닙니다. O 로 찍어도 되게 해 달라는 겁니다

같은 나라, 같은 연구, 다른 규정

대학 연구실에서는

- 서울대 장혜식 교수 연구실 — 랩 Notion 의 매일 diff 를 LLM 이 요약해 Slack 으로 보냅니다.
Claude Code 로 4시간
- 지난주 어느 대학 수업 — 랩 네트워크 시뮬레이터에 **Tailscale** 로 붙어 **라이브**로 보여주더군요

제 기관에서는

- Notion · Slack · GitHub → **외부 SaaS**
- Tailscale → **사설 VPN**
- 전부 승인 대상입니다. 그래서 **대개 안 합니다**

⚠ **Tailscale** 을 쓰는 이유가 보안입니다 — 포트를 열지 않아도 되고, 신원으로 막습니다. 22번 열어두는 것보다 안전합니다. 그런데 규정은 그걸 **「사설 VPN」**으로 분류합니다

기술도 사람도 같습니다. 다른 건 규정 하나입니다.
그리고 더 안전한 쪽이 더 비싼 절차를 칩니다

비어 있는 다섯 층

비어 있는 층	채우면
1 분류 판정 기준표	천체 관측 데이터는? 투고 전 원고는? 실험실 리눅스는? <i>예시 시나리오 30개면 됩니다</i>
2 분류자 보호 조항	절차대로 근거 남기고 분류했으면 결과에 면책 <i>없으면 아무도 O^{공개} 를 못 씁니다 → ㉔</i>
3 등급별 최소 통제 참조 구현	문장이 아니라 실제 설정 으로
4 질의 답변 공개 누적	판례집 이 쌓여야 매번 처음부터 안 합니다
5 재분류 · 이의 절차	한 번 매긴 등급을 되돌릴 길

? 현장 담당자는 이 문장을 어떻게 실행합니까?

이 층은 원래 현장이 채우는 층입니다

바뀌야 할 건 기술이 아니라 가격표입니다

지금 제도는 금지의 비용을 0으로, 구축의 비용을 사업으로 매깁니다.

제안 아홉 가지

제안	소관
① 총량제 집계 단위를 관리 주체 수 로 · 폐지 도메인은 301 유지	행정안전부 가이드
② 정적 호스팅 은 별도 등급으로	☐
③ 제출물은 설명이 아니라 저장소 주소 하나	☐
④ 검토 주기와 관측을 교환	기관 정보보안 내규
⑤ 긴급 보안 조치에 만료일	기관 정보보안 내규
⑥ 분류자 보호 — 절차대로 분류했으면 결과에 면책	☐
⑦ 기반 인프라에 존속 KPI	☐
⑧ 국가 오픈소스 인프라 전담 조직 과 공식 미러	부처 직제·예산
⑨ 개편 전 고지 · 납품 요건에 반영	☐

아홉 중 여섯은 법이 아니라 지침과 내규입니다. 오늘 시작할 수 있습니다

① 총량제 집계 단위를 바꿔주십시오

- **현행** — 호스트명(서브도메인) 수로 집계
- **문제** — 서브도메인 분리는 **브라우저의 보안 경계**입니다. 서브URI로 묶으면 origin을 공유해 쿠키·세션이 섞이고, 한 앱의 XSS가 나머지를 가져갑니다
- **제안 ㉠** — **독립적으로 운영·패치·폐기되는 관리 주체** 수로
- **제안 ㉡** — 폐지 도메인은 반납이 아니라 **경로를 보존하는 301 + 유효한 인증서** 유지 의무로
- **소관** — 행정안전부 가이드 개정 (*법 개정 아님*)

× 3부에서 보신 그 플랫폼은 앱마다 서브도메인을 발급합니다.
총량제 밑에서는 — **안전하게 만들수록 총량을 먹습니다**

지금 지표는 안전한 구조에 벌점을 줍니다

② 정적 호스팅은 별도 등급으로

- 서버 사이드 로직 없음 · DB 없음 · 사용자 입력 없음 · **전부 공개 예정 정보**
- 침해 시 피해는 **콘텐츠 변조. 유출할 데이터가 존재하지 않습니다**
- 그런데 현행은 동적 웹 서비스와 **동일한 절차**를 요구합니다

털릴 게 없는 것을 지키느라 털릴 게 있는 것을 못 보고 있습니다

정직하게 붙일 단서 둘 — ① 변조 위험은 남습니다(기관 도메인 정적 페이지 = 피싱 랜딩). 대응은 Git 자동 복구. ② 폼 하나·외부 API 하나 붙으면 등급이 달라집니다. **정적 판정 기준**을 함께 제안해야 합니다.

③ 제출물은 설명이 아니라 저장소 주소 하나

현행	제안
사람이 산출물을 설명	Git 저장소 주소 하나
신청서 · 화면 캡처	기관 CI가 자동 판정 정적 여부 · 시크릿 · 개인정보 패턴 · 빌드 재현
담당자가 읽고 판단	사전 심사 면제 · 커밋 로그가 증거
승인 후엔 뭘 바꾸든 아무도 모름	변경 시 자동 재검사

일회 심사보다 오히려 강합니다 — 상시니까요

④ 검토 주기와 관측을 교환해주십시오

- 해당 조건을 충족한 팀은 **심사빈도 완화** ($1/4$ 정도로)
 - 검증된 기본 통제 / 지정된 담당자 / 관측 가능성(로그 상시 전송) / 복구 가능성
- 완화되는 것은 **심사 빈도지 운영 책임이 아닙니다**
- 로그 전송이 끊기면 **자동으로 원래 주기로 복귀합니다**
- **소관** — 기관 정보보안 내규 (기관장 전결 가능)

금지 대신 관측에 보상을 붙이는 것 — 인센티브를 뒤집는 유일한 방법

⑤ 긴급 보안 조치에 만료일을 붙여주십시오

2001년 그 공지에 만료일과 재검토 의무가 있었다면 오늘 우리가 이려고 있지 않습니다

- 가장 싸고 가장 근본적인 조항입니다
- 소관 — 기관 정보보안 내규

⑥ 분류자 보호 조항

- 절차대로 근거를 남기고 분류했으면, 결과에 면책
- 기준 정합(국정원 · 감사원)은 부처가 달라 오래 걸립니다. 면책은 지침 한 줄입니다
- 소관 — ☐

O 로 찍은 사람을 보호하지 않으면, 아무도 O 를 쓰지 않습니다

⑦ 기반 인프라에 존속 KPI

- 지표는 셋이면 됩니다 — 보존 아카이브 규모 · 외부 참조 수 · 미러 제공 트래픽
- 소관 — ☐

× 조직이 있어도 KPI 가 없으면 또 사라집니다

1부의 `ftp.kreonet.re.kr` 이 그 증거입니다 — 예산이 없어서가 아니라, 그 일을 계속할 이유가 아무에게도 없었기 때문입니다

인프라 운영을 성과로 인정하는 트랙이 필요합니다 — 예산이 없어서가 아니라
경력이 안 되기 때문입니다

⑧ 국가 오픈소스 인프라 전담 조직과 공식 미러

- 모델은 대만 NCHC 자유소프트웨어실험실 — 미러를 운영하는 팀이 도구를 만듭니다
- 둘은 같은 역량입니다. 그리고 인원 없이는 남의 것을 가져다 쓰는 것조차 안 됩니다
- 슈퍼컴과 연구망을 동시에 가진 기관은 국내에 하나뿐입니다
- 내놓을 것은 미러만이 아닙니다 — 레퍼런스 · 교육 · 핵심 부품 점검입니다. Trusted CI 가 Framework · OSCRIP · 1:1 협력을 내놓듯이
- 한국 미러 목록에 기관이 하나도 없습니다 — 기업 · 개인 · 학생 동아리뿐입니다
- 소관 — 부처 직제·예산 (지침 개정으로는 안 되는 유일한 항목)

i 유출을 막는 건 맞습니다. 기술 유출은 민감하고, 외부 메일이 되는 곳은 이미 그만큼 보안 범위가 정해진 겁니다. 문제는 연속성과 협업입니다 — 안 해주면 바깥 것을 찾습니다. 필요한 건 완화가 아니라 안전하게 구축하고 쓰는 법에 대한 교육과 레퍼런스입니다

가져다 쓰는 것도 사람이 있어야 합니다

대만 NCHC 자유소프트웨어실험실 — 국가 미러 free.nchc.org.tw(CentOS · Ubuntu · Arch · openSUSE · Kali)를 운영하면서 같은 팀이 Clonezilla · DRBL 을 만듭니다 · 일본은 RIKEN 니시나센터가 ftp.riken.jp 우리도 있습니다 — 과기부·NIPA oss.kr, 「오픈소스 AI·SW 개발·활용 지원사업」. 다만 이미 쓰고 있는 것을 고치는 쪽은 아닙니다

⑧ 그럼 이 부품들은 누가 봅니까

참조구현을 퍼뜨리는 순간, 그 부품이 모든 기관의 급소가 됩니다

부품	터지면
Caddy 리버스 프록시 · TLS	입구가 통째로
oauth2-proxy · authentik OIDC	인증이 통째로
BIND9 DNS-01 챌린지	인증서 발급이 통째로
squid 허용목록 프록시	빌드 경로가 통째로
Docker · containerd · RKE2	격리가 통째로

! 부품 하나에 취약점 소식이 나면 그 위에 선 전부가 흔들립니다.
이 참조구현은 **Caddy** 로 짰습니다 — 그럼 **Caddy** 가 안 터져야 합니다. **DNS-01** 을 쓰기로 했으면 **BIND9** 도 안전하게 관리돼야 합니다

Trusted CI 가 Jupyter · Singularity · Tapis 에 한 일이 정확히 이겁니다

㉠ 개편 전에 알려주시고, 납품 요건에 넣어주십시오

i (가) 개편 예고

시스템을 바꾸기 전 **30일 전** 공지와 바뀌는
엔드포인트 목록 공개

i (나) 납품 요건

기관이 발주하는 내부 시스템에 **읽기 전용 조회**
API 제공을 요건으로

API를 새로 만들어 달라는 게 아닙니다.

깨뜨릴 거면 깨진다고 알려만 달라는 것입니다. 비용이
거의 없습니다

발주 문구 한 줄입니다.

지금 계약하는 것부터 달라집니다

- 기존 시스템에 조회 API를 다는 건 사업입니다. **그건 장기 과제로 두겠습니다**
- 위 둘은 사업이 아닙니다. 오늘 결재선에서 됩니다

"큰 사업이 아니면 아무것도 안 한다" — 그 습관을 이 제안에서는 저희도 안 쓰겠습니다

✓ **아홉 중 여섯은 법이 아니라 지침과 내규입니다. 오늘 시작할 수 있습니다**

보론 — 하나로 같까요, 여럿으로 같까요

고정해야 할 것

열어둬야 할 것

4개 통제 지점 정의

감사 로그 스키마

증적 산출물 포맷

적합성 검증 테스트

구현

k8s SADP / docker compose 경량판 / 기존 인프라에 얹기

전자정부 프레임워크가 실패한 건 **하나여서가 아니라 구현을 고정했기** 때문입니다. 아까 본 **Trusted CI Framework** 가 그 반대입니다 — **충족 요건만 정하고 구현은 안 묻습니다.**

감사합니다

장민석 · KISTI 과학기술연구망센터 · msjang@kisti.re.kr

자료 — msjang.github.io/ostday26-sadp

